Contents

Market Basket Analysis with Python: Snapdeal

The aim is to analyze customer purchasing behavior, product affinities, and engagement trends to enhance personalized shopping experiences and optimize inventory and marketing strategies. The focus is to understand:

- What products or categories are frequently purchased together, indicating cross-selling opportunities?
- How do customer purchasing patterns vary across demographics or regions?
- Can we segment customers based on buying behavior to target personalized promotions?
- Which products drive repeat purchases or high customer loyalty?

Project Objective:

Build a comprehensive Customer Purchasing Behavior and Product Recommendation Report using Python for data analysis. This report should clean and consolidate Snapdeal's transaction and customer data, apply techniques such as Market Basket Analysis, clustering, and association rule mining, and provide actionable insights for personalized product recommendations, promotional strategies and inventory optimization.

Datasets Available:

1. **SnapDeal.csv:**
   i) Timestamp: Timestamp of survey submission
   ii) Age: Age of the respondent
   iii) Gender: Gender identity of the respondent
   iv) Purchase_Frequency: How often the respondent shops on Snapdeal
   v) Purchase_Categories: Product categories the respondent buys
   vi) Personalized_Recommendation_Frequency: How often personalized recommendations are shown
   vii) Browsing_Frequency: How often the respondent browses Snapdeal
   viii) Browsing_Duration; Average browsing duration per session
   ix) Browsing_Device: Device used for browsing Snapdeal
   x) Search_Frequency: Frequency of using the search feature
   xi) Search_Filter_Usage: How often filters are applied during search
   xii) Search_Result_Exploration: Depth of exploration of search results pages
   xiii) Customer_Reviews_Importance: Importance of customer reviews in decision-making
   xiv) Add_to_Cart_Browsing: Whether users add items to cart for browsing later
   xv) Cart_Completion_Frequency: How often users complete purchases from cart
   xvi) Cart_Abandonment_Factors: Reasons for abandoning items in cart
   xvii) Saveforlater_Frequency: Frequency of using Save for Later feature

xviii) Review_Left: Whether the respondent leaves reviews after purchase
xix) Review_Reliability: Level of trust in customer reviews
xx) Review_Helpfulness: Whether reviews are found helpful
xxi) Personalized_Recommendation_Frequency_2: Duplicate column (cleaned)
xxii) Recommendation_Helpfulness: Helpfulness of personalized recommendations
xxiii) Rating_Accuracy: Trust in the accuracy of product ratings
xxiv) Shopping_Satisfaction: Overall shopping satisfaction with Snapdeal
xxv) Service_Appreciation: Appreciation of Snapdeal's customer service
xxvi) Improvement_Areas: Suggestions for improvement
xxvii) Transaction: Randomly generated transaction ID column

Technical Work Documentation

Task 1: Data Cleaning and Preparation

1. Removed duplicate or inconsistent survey responses.

```
# TASK 1: DATA CLEANING AND PREPARATION


# Task 1.1: Removing duplicate or inconsistent survey responses

# Checking for duplicate rows
duplicate_rows = data.duplicated().sum()
print('Number of duplicate rows = ', duplicate_rows)
```

```
Number of duplicate rows =  0
```

```
# Checking for duplicate transaction ids
duplicate_id = data['transaction'].duplicated().sum()
print('Number of duplicate transactions = ', duplicate_id)
```

```
Number of duplicate transactions =  0
```

2. Standardized categorical/numerical entries (e.g., frequency levels, gender, recommendation responses).
   Created cat_col which includes those columns with string data types and found unique values after dropping any missing values.

```
# Task 1.2: Checking for inconsistency in textual/categorical columns
cat_col = data.select_dtypes(include = 'object').columns
for i in cat_col:
  print(f'\n {i}')
  print(data[i].dropna().unique())
```

```
 'Beauty and Personal Care;Home and Kitchen;others'
 'Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion;others'
 'Home and Kitchen' 'Groceries and Gourmet Food;Home and Kitchen'
 'Groceries and Gourmet Food;Clothing and Fashion']

 Personalized_Recommendation_Frequency
['Yes' 'Sometimes' 'No']

 Browsing_Frequency
['Few times a month' 'Rarely' 'Few times a week' 'Multiple times a day']

 Product_Search_Method
```

   Removed leading and trailing spaces from the values.

```
# Removing leading & trailing spaces in categorical columns (Like Service_Appreciation)
for i in cat_col:
  data[i] = data[i].str.strip()
```

```
print(data['Service_Appreciation'].unique())
print(data['Improvement_Areas'].unique())
```

```
['Customer service' '.' 'Wide product selection' 'All the above'
 'User-friendly website/app interface' 'Quick delivery'
 'Competitive prices' 'Product recommendations']
['User interface of app' '.' 'Product quality and accuracy'
 "I don't have any problem with Amazon" 'Customer service responsiveness'
 'Nil' 'No problems with Amazon' 'Irrelevant product suggestions'
 'better app interface and lower shipping charges'
 'Shipping speed and reliability' 'Nothing'
 'I have no problem with Amazon yet. But others tell me about the refund issues'
 'UI' 'Quality of product is very poor according to the big offers'
 'Scrolling option would be much better than going to next page'
 'User interface' 'Reducing packaging waste'
 'Add more familiar brands to the list']
```

3. Handled missing values and inconsistent formats in Product_Search_Method and other fields.
   Found out that there are 149 missing (nan) values in Product_Search_Method. Also converted all existing dot values to nan to remove all nan values collectively at once.

```
# Task 1.3: Handling missing values and inconsistent formats
#           in Product_Search_Method and other fields.
miss_val = data.isnull().sum()
print('Number of missing Values = ', miss_val[miss_val > 0])
# Checking the number of '.' values
dot_val = (data == '.').sum()
print('Number of dot values = ', dot_val[dot_val > 0])
```

```
Number of missing Values =  Product_Search_Method    149
dtype: int64
Number of dot values =  Service_Appreciation    89
Improvement_Areas        38
dtype: int64
```

```
# Converting all '.' values to nan to collectively remove
# all nan values together in the next step
import numpy as np
data = data.replace('.', np.nan)
print(data.isnull().sum()[data.isnull().sum() > 0])
```

```
Product_Search_Method    149
Service_Appreciation      89
Improvement_Areas         38
dtype: int64
```

```
# See the distribution of Product_Search_Method
print(data['Product_Search_Method'].value_counts(dropna=False))
```

```
Product_Search_Method
others         182
Keyword        163
categories     156
Filter         150
NaN            149
Name: count, dtype: int64
```

```
# Total missing values after converting '.' to nan
print("Total missing values:", data.isnull().sum().sum())
# Since it makes up 34.5% of the total data, we won't drop these

Total missing values: 276
```

Since the total number of nan values in our dataset makes up around 34.5%, we did not remove them.

4. Renamed duplicate or misformatted columns (e.g., remove trailing spaces in Rating_Accuracy).
   Since there are two columns with the same name, checked their values. Found out that both have different values where one consists of frequency while the other consists of ratings. Renamed the second column name as Personalized_Recommendation_Frequency_Rating and removed trailing space in Rating_Accuracy column name.

```
print(data.columns.tolist())

['Timestamp', 'age', 'Gender', 'Purchase_Frequency', 'Purchase_Categories', 'Pers

# Since there are 2 columns with the same names
print(data['Personalized_Recommendation_Frequency'].value_counts(dropna=False))
print(data['Personalized_Recommendation_Frequency '].value_counts(dropna=False))

Personalized_Recommendation_Frequency
No           285
Sometimes    272
Yes          243
Name: count, dtype: int64
Personalized_Recommendation_Frequency
1    173
2    166
3    163
4    155
5    143
Name: count, dtype: int64
```

```
# Renaming the 2nd same name column
# and removing trailing space in Rating_Accuracy
data = data.rename(columns = {'Personalized_Recommendation_Frequency ':
                              'Personalized_Recommendation_Frequency_Rating',
                              'Rating_Accuracy ': 'Rating_Accuracy'})
```

5. Converted numerical rating columns (e.g., Customer_Reviews_Importance, Shopping_Satisfaction) to appropriate numeric types for analysis.
   Created a list of numeric rating columns first and validated that all are numeric.

```
# Task 1.5: Convert numerical rating columns
#           (e.g., Customer_Reviews_Importance, Shopping_Satisfaction)
#           to appropriate numeric types for analysis

# Finding the numeric rating columns
rate_col = ['Customer_Reviews_Importance', 'Personalized_Recommendation_Frequency_Rating',
            'Recommendation_Helpfulness', 'Rating_Accuracy', 'Shopping_Satisfaction', 'Review_Helpfulness']
print(data[rate_col].dtypes)
```

```
Customer_Reviews_Importance                  int64
Personalized_Recommendation_Frequency_Rating     int64
Recommendation_Helpfulness                  object
Rating_Accuracy                              int64
Shopping_Satisfaction                        int64
Review_Helpfulness                          object
dtype: object
```

```
# Validating that all numeric rating columns are integers
rate_col = ['Customer_Reviews_Importance',
            'Personalized_Recommendation_Frequency_Rating',
            'Rating_Accuracy','Shopping_Satisfaction']
for i in rate_col:
    print(f"{i}:")
    print("Data type:", data[i].dtype)
    print("Unique values:", sorted(data[i].dropna().unique()))
    print()
```

```
Customer_Reviews_Importance:
Data type: int64
Unique values: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]

Personalized_Recommendation_Frequency_Rating:
Data type: int64
Unique values: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]

Rating_Accuracy:
Data type: int64
Unique values: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
Shopping_Satisfaction:
Data type: int64
Unique values: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
```

Task 2: Descriptive Behavioral Analysis

1. Summarized customer demographics (age, gender distribution)
   Found out Age and Gender statistics and percentage contribution of each gender among the respondents.

```
# Task 2.1: Summarizing customer demographics (age, gender distribution)
print('Age Statistics')
print(data['age'].describe())
print('Gender Statistics')
print(data['Gender'].describe())
print('Gender %')
print(data['Gender'].value_counts(normalize = True) * 100)
```

```
Age Statistics
count    800.000000
mean      36.053750
std       19.310087
min        3.000000
25%       18.000000
50%       37.000000
75%       53.000000
max       67.000000
Name: age, dtype: float64
Gender Statistics
count      800
unique       4
top       Male
freq       213
Name: Gender, dtype: object
Gender %
Gender
Male                26.625
Female              25.125
Prefer not to say   24.250
Others              24.000
```

Therefore, the middle 50% age groups are from 18-53 years. The largest gender group is of Male (26.63%) but it does not differ significantly from other gender percentages.

2. Analyzed overall purchase frequency and most popular product categories.

```
# Task 2.2: Analyzing overall purchase frequency
#           and most popular product categories
print('Purchase Frequency')
print(data['Purchase_Frequency'].value_counts())
print('Purchase Frequency %')
print(data['Purchase_Frequency'].value_counts(normalize = True)*100)
print('Purchase Categories')
print(data['Purchase_Categories'].value_counts().head(5))
```

```
Purchase_Frequency
Multiple times a week     166
Once a month              164
Once a week               161
Few times a month         157
Less than once a month    152
Name: count, dtype: int64
Purchase Frequency %
Purchase_Frequency
Multiple times a week     20.750
Once a month              20.500
Once a week               20.125
Few times a month         19.625
Less than once a month    19.000
Name: proportion, dtype: float64
Purchase Categories
Purchase_Categories
Clothing and Fashion                                                                  36
Clothing and Fashion;Home and Kitchen                                                 34
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion              34
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion;Home and Kitchen    32
others                                                                                32
```

Found out individual product categories and their percentages.

```
# Individual Categories Count and Percentage of respondents
# who selected each category
cat_count = data['Purchase_Categories'].dropna().str.split(';').explode().str.strip().value_counts()
cat_percent = cat_count/len(data) * 100
print(cat_count)
print(cat_percent)
```

```
Purchase_Categories
Clothing and Fashion          476
Beauty and Personal Care      414
Home and Kitchen              405
others                        396
Groceries and Gourmet Food    381
Name: count, dtype: int64
Purchase_Categories
Clothing and Fashion          59.500
Beauty and Personal Care      51.750
Home and Kitchen              50.625
others                        49.500
Groceries and Gourmet Food    47.625
Name: count, dtype: float64
```

3. Identified top browsing methods and most common cart abandonment factors. Calculating the contribution of different values in browsing frequency, product search method, search result exploration, add to cart browsing, cart completion frequency and cart abandonment factors in numbers using value_counts() and percentages using value_counts(normalize = True).

```python
# Analyzing Browsing Frequency Firstly
print('Browsing Frequency')
print(data['Browsing_Frequency'].value_counts())
print('Browsing Frequency %')
print(data['Browsing_Frequency'].value_counts(normalize = True)*100)
```

```
Browsing Frequency
Browsing_Frequency
Multiple times a day    204
Few times a week        202
Few times a month       199
Rarely                  195
Name: count, dtype: int64
Browsing Frequency %
Browsing_Frequency
Multiple times a day    25.500
Few times a week        25.250
Few times a month       24.875
Rarely                  24.375
Name: proportion, dtype: float64
```

```python
# Analyzing Product Search Methods
print('Product Search Method')
print(data['Product_Search_Method'].value_counts(dropna = False))
print('Product Search Method %')
print(data['Product_Search_Method'].value_counts(normalize = True, dropna = False)*100)
```

```
Product Search Method
Product_Search_Method
others        182
Keyword       163
categories    156
Filter        150
NaN           149
Name: count, dtype: int64
Product Search Method %
Product_Search_Method
others        22.750
Keyword       20.375
categories    19.500
Filter        18.750
NaN           18.625
Name: proportion, dtype: float64
```

```python
# Analyzing Search Result Exploration
print('Search Result Exploration')
print(data['Search_Result_Exploration'].value_counts())
print('Search Result Exploration %')
print(data['Search_Result_Exploration'].value_counts(normalize = True)*100)
```

```
Search Result Exploration
Search_Result_Exploration
Multiple pages    416
First page        384
Name: count, dtype: int64
Search Result Exploration %
Search_Result_Exploration
Multiple pages    52.0
First page        48.0
Name: proportion, dtype: float64
```

```python
# Analyzing Add to Cart Browsing
print('Add to Cart Browsing')
print(data['Add_to_Cart_Browsing'].value_counts())
print('Add to Cart Browsing %')
print(data['Add_to_Cart_Browsing'].value_counts(normalize = True)*100)
```

```
Add to Cart Browsing
Add_to_Cart_Browsing
Yes      276
No       265
Maybe    259
Name: count, dtype: int64
Add to Cart Browsing %
Add_to_Cart_Browsing
Yes      34.500
No       33.125
Maybe    32.375
Name: proportion, dtype: float64
```

```python
# Analyzing Cart Completion Frequency
print('Cart Completion Frequency')
print(data['Cart_Completion_Frequency'].value_counts())
print('Cart Completion Frequency %')
print(data['Cart_Completion_Frequency'].value_counts(normalize = True)*100)
```

```
Cart Completion Frequency
Cart_Completion_Frequency
Often        173
Always       166
Sometimes    158
Never        157
Rarely       146
Name: count, dtype: int64
Cart Completion Frequency %
Cart_Completion_Frequency
Often        21.625
Always       20.750
Sometimes    19.750
Never        19.625
Rarely       18.250
```

```python
# Analyzing Cart Abandoment Factors
print('Cart Abandonment Factors')
print(data['Cart_Abandonment_Factors'].value_counts())
print('Cart Abandonment Factors %')
print(data['Cart_Abandonment_Factors'].value_counts(normalize = True)*100)
```

```
Cart Abandonment Factors
Cart_Abandonment_Factors
Found a better price elsewhere                 219
others                                         196
High shipping costs                            194
Changed my mind or no longer need the item     191
Name: count, dtype: int64
Cart Abandonment Factors %
Cart_Abandonment_Factors
Found a better price elsewhere                 27.375
others                                         24.500
High shipping costs                            24.250
Changed my mind or no longer need the item     23.875
Name: proportion, dtype: float64
```

4. Calculated mean and median satisfaction, recommendation helpfulness, and rating accuracy using describe and value_counts() functions.

```
# Task 2.4: Calculating mean and median satisfaction,
#           recommendation helpfulness, and rating accuracy
print('Shopping Satisfaction')
print(data['Shopping_Satisfaction'].describe())
print('Rating Accuracy')
print(data['Rating_Accuracy'].describe())
print('Recommendation Helpfulness')
print(data['Recommendation_Helpfulness'].value_counts(normalize = True)*100)
print("Personalized Recommendation Frequency Rating:")
print(data['Personalized_Recommendation_Frequency_Rating'].describe())
```

```
Shopping Satisfaction
count    800.000000
mean       2.988750
std        1.407515
min        1.000000
25%        2.000000
50%        3.000000
75%        4.000000
max        5.000000
Name: Shopping_Satisfaction, dtype: float64
Rating Accuracy
count    800.00000
mean       2.96375
std        1.42565
min        1.00000
25%        2.00000
50%        3.00000
75%        4.00000
max        5.00000
Name: Rating_Accuracy, dtype: float64
Recommendation Helpfulness
Recommendation_Helpfulness
No           36.875
Yes          32.625
Sometimes    30.500
Name: proportion, dtype: float64
Personalized Recommendation Frequency Rating:
count    800.000000
mean       2.911250
std        1.405647
min        1.000000
25%        2.000000
50%        3.000000
75%        4.000000
max        5.000000
Name: Personalized_Recommendation_Frequency_Rating, dtype: float64
```

5. Generated summary statistics and visualizations for key behavioral variables. Created a summary dataframe consisting of statistical summarization of the behavioral data. Visualizations covered later.

```
# Task 2.5: Generating summary statistics and visualizations for key behavioral variables.
summary = data[['Shopping_Satisfaction', 'Rating_Accuracy',
                'Personalized_Recommendation_Frequency_Rating']].agg(
                    ['mean', 'median', 'std', 'min', 'max']).T
print(summary)
```

```
                                           mean  median       std  min  \
Shopping_Satisfaction                   2.98875     3.0  1.407515  1.0
Rating_Accuracy                         2.96375     3.0  1.425650  1.0
Personalized_Recommendation_Frequency_Rating  2.91125     3.0  1.405647  1.0

                                           max
Shopping_Satisfaction                      5.0
Rating_Accuracy                            5.0
Personalized_Recommendation_Frequency_Rating  5.0
```

Task 3: Customer Segmentation and Profiling

1. Segmented customers based on purchase frequency and shopping satisfaction levels.
2. Create profiles such as:
   Frequent Buyers: High purchase frequency, high satisfaction.
   Occasional Shoppers: Medium frequency, moderate satisfaction.
   At-Risk Customers: Low satisfaction or frequent cart abandonment.
   Purchase Frequency -
   i) Multiple times a week or once a week: High Frequency

ii) Few times a month or once a month: Medium Frequency

iii) Less than once a month: Low Frequency

Shopping Satisfaction -

i) 1-2 Rating: Low Satisfaction

ii) 3 Rating: Medium Satisfaction

iv) 4-5 Rating: High Satisfaction

```
# Task 3.1: Segmenting customers based on purchase frequency
#          & shopping satisfaction levels

# Task 3.2: Create profiles such as:
# Frequent Buyers: High purchase frequency, high satisfaction
# Occasional Shoppers: Medium frequency, moderate satisfaction
# At-Risk Customers: Low satisfaction or frequent cart abandonment
segment_table = pd.crosstab(data['Purchase_Frequency'],
                            data['Shopping_Satisfaction'])
print(segment_table)
```

```
Shopping_Satisfaction    1   2   3   4   5
Purchase_Frequency
Few times a month       36  31  36  25  29
Less than once a month  21  30  36  32  33
Multiple times a week   33  36  35  26  36
Once a month            36  35  37  26  30
Once a week             30  32  33  30  36
```

Creating columns called Purchase_Frequency_Segment and Satisfaction_Segment.

```
# Creating both columns
data['Purchase_Frequency_Segment'] = data['Purchase_Frequency'].map({
    'Multiple times a week': 'High Frequency',
    'Once a week': 'High Frequency',
    'Few times a month': 'Medium Frequency',
    'Once a month': 'Medium Frequency',
    'Less than once a month': 'Low Frequency'
    })
data['Satisfaction_Segment'] = pd.cut(
    data['Shopping_Satisfaction'],
    bins = [0,2,3,5],
    labels = ['Low Satisfaction', 'Medium Satisfaction', 'High Satisfaction'])
print(data[['Purchase_Frequency', 'Purchase_Frequency_Segment',
            'Shopping_Satisfaction', 'Satisfaction_Segment']].head(10))
```

Now checking the segments distribution.

```
# Checking the segment distribution
print(data['Purchase_Frequency_Segment'].value_counts())
print('Purchase Frequency Segment %')
print(data['Purchase_Frequency_Segment'].value_counts(normalize = True)*100)
print(data['Satisfaction_Segment'].value_counts())
print('Satisfaction Segment %')
print(data['Satisfaction_Segment'].value_counts(normalize = True)*100)
```

```
Purchase_Frequency_Segment
High Frequency      327
Medium Frequency    321
Low Frequency       152
Name: count, dtype: int64
Purchase Frequency Segment %
Purchase_Frequency_Segment
High Frequency      40.875
Medium Frequency    40.125
Low Frequency       19.000
Name: proportion, dtype: float64
Satisfaction_Segment
Low Satisfaction       320
High Satisfaction      303
Medium Satisfaction    177
Name: count, dtype: int64
Satisfaction Segment %
Satisfaction_Segment
Low Satisfaction       40.000
High Satisfaction      37.875
Medium Satisfaction    22.125
Name: proportion, dtype: float64
```

Therefore, low satisfaction customers are the highest with 40% and high frequency customers are the highest.

Now, all the 3 levels of both the segments are combined to distribute the customers into 9 groups depending upon satisfaction levels and frequency segments where Medium frequency - Low satisfaction group forms the highest segment.

```
# Combining all the 6 segments to get total 9 combinations
data['Cust_Segment'] = data['Purchase_Frequency_Segment']+'-'+data['Satisfaction_Segment'].astype(str)
print(data['Cust_Segment'].value_counts())
```

```
Cust_Segment
Medium Frequency-Low Satisfaction       138
High Frequency-Low Satisfaction         131
High Frequency-High Satisfaction        128
Medium Frequency-High Satisfaction      110
Medium Frequency-Medium Satisfaction     73
High Frequency-Medium Satisfaction       68
Low Frequency-High Satisfaction          65
Low Frequency-Low Satisfaction           51
Low Frequency-Medium Satisfaction        36
Name: count, dtype: int64
```

```
# Percentage Segmentation
segment_summary = pd.DataFrame({'Count': data['Cust_Segment'].value_counts(),
                                'Percentage': data['Cust_Segment'].value_counts
                                 (normalize=True) * 100}).sort_values('Count',
                                                                      ascending = True)
print(segment_summary)
```

```
                                  Count  Percentage
Cust_Segment
Low Frequency-Medium Satisfaction    36       4.500
Low Frequency-Low Satisfaction       51       6.375
Low Frequency-High Satisfaction      65       8.125
High Frequency-Medium Satisfaction   68       8.500
Medium Frequency-Medium Satisfaction 73       9.125
Medium Frequency-High Satisfaction  110      13.750
High Frequency-High Satisfaction    128      16.000
High Frequency-Low Satisfaction     131      16.375
Medium Frequency-Low Satisfaction   138      17.250
```

Created profile for frequent buyers (high purchase frequency, high satisfaction), occasional buyers (medium frequency, moderate satisfaction) and at-risk customers (low satisfaction or frequent cart abandonment).

```
# Task 3.2: Create profiles such as:
# Frequent Buyers: High purchase frequency, high satisfaction
# Occasional Shoppers: Medium frequency, moderate satisfaction
# At-Risk Customers: Low satisfaction or frequent cart abandonment
frequent_buyers = data[(data['Purchase_Frequency_Segment'] == 'High Frequency')&
                       (data['Satisfaction_Segment'] == 'High Satisfaction')]
occasional_shoppers = data[(data['Purchase_Frequency_Segment'] == 'Medium Frequency')&
                           (data['Satisfaction_Segment'] == 'Medium Satisfaction')]
at_risk = data[(data['Satisfaction_Segment'] == 'Low Satisfaction')|
               (data['Cart_Completion_Frequency'].isin(['Rarely', 'Never']))]
```

3. Analysed demographic or behavioural differences across these segments. Found out age wise and gender wise differences across the 3 buyer profiles. Also found out the cart completion behaviour across the profiles.

```
print("Average Age")
print("Frequent Buyers:", frequent_buyers['age'].mean())
print("Occasional Shoppers:", occasional_shoppers['age'].mean())
print("At-Risk Customers:", at_risk['age'].mean())
print('\n')

# Gender wise differences across the profiles
gender_profile = pd.DataFrame({
    'Frequent Buyers': frequent_buyers['Gender'].value_counts(normalize = True)*100,
    'Occasional Shoppers': occasional_shoppers['Gender'].value_counts(normalize = True)*100,
    'At-Risk Customers': at_risk['Gender'].value_counts(normalize = True)*100
}).fillna(0)
print(gender_profile)
print('\n')

# Cart completion behaviour across the profiles
cart_profile = pd.DataFrame({
    'Frequent Buyers': frequent_buyers['Cart_Completion_Frequency'].value_counts(normalize = True)*100,
    'Occasional Shoppers': occasional_shoppers['Cart_Completion_Frequency'].value_counts(normalize = True)*100,
    'At-Risk Customers': at_risk['Cart_Completion_Frequency'].value_counts(normalize = True)*100})
```

```
Average Age
Frequent Buyers: 35.984375
Occasional Shoppers: 37.57534246575342
At-Risk Customers: 35.662


                  Frequent Buyers  Occasional Shoppers  At-Risk Customers
Gender
Female                  21.09375            26.027397               25.2
Male                    28.12500            34.246575               25.0
Others                  27.34375            19.178082               24.6
Prefer not to say       23.43750            20.547945               25.2
```

Cart_Completion_Frequency	Frequent Buyers	Occasional Shoppers	At-Risk Customers
Always	23.43750	19.178082	14.4
Never	21.87500	23.287671	31.4
Often	19.53125	24.657534	11.4
Rarely	18.75000	20.547945	29.2
Sometimes	16.40625	12.328767	13.6

4. Used clustering (e.g., K-Means) for behavioural grouping based on survey responses.
   Created clust_col consisting of the required behavioural columns.

```
# Task 3.4: Using clustering (e.g. K-Means) for behavioral grouping
#           based on survey responses
clust_col = ['Purchase_Frequency','Browsing_Frequency','Product_Search_Method',
             'Search_Result_Exploration','Add_to_Cart_Browsing','Cart_Completion_Frequency',
             'Saveforlater_Frequency','Recommendation_Helpfulness','Review_Left',
             'Review_Reliability','Review_Helpfulness','Customer_Reviews_Importance',
             'Personalized_Recommendation_Frequency_Rating','Rating_Accuracy',
             'Shopping_Satisfaction']

for i in clust_col:
    print(f"\n{i}:")
    print(data[i].value_counts(dropna=False))
```

```
Purchase_Frequency:
Purchase_Frequency
Multiple times a week    166
Once a month             164
Once a week              161
Few times a month        157
Less than once a month   152
Name: count, dtype: int64

Browsing_Frequency:
Browsing_Frequency
Multiple times a day    204
Few times a week        202
Few times a month       199
Rarely                  195
Name: count, dtype: int64
```

```
Product_Search_Method:
Product_Search_Method
others          182
Keyword         163
categories      156
Filter          150
NaN             149
Name: count, dtype: int64

Search_Result_Exploration:
Search_Result_Exploration
Multiple pages    416
First page        384
Name: count, dtype: int64

Add_to_Cart_Browsing:
Add_to_Cart_Browsing
Yes      276
No       265
Maybe    259
Name: count, dtype: int64
```

```
Cart_Completion_Frequency:
Cart_Completion_Frequency
Often        173
Always       166
Sometimes    158
Never        157
Rarely       146
Name: count, dtype: int64

Saveforlater_Frequency:
Saveforlater_Frequency
Sometimes    199
Always       160
Never        157
Rarely       147
Often        137
Name: count, dtype: int64
```

```
Saveforlater_Frequency:
Saveforlater_Frequency
Sometimes    199
Always       160
Never        157
Rarely       147
Often        137
Name: count, dtype: int64

Recommendation_Helpfulness:
Recommendation_Helpfulness
No           295
Yes          261
Sometimes    244
Name: count, dtype: int64

Review_Left:
Review_Left
Yes    401
No     399
Name: count, dtype: int64
```

Likewise, we get the outputs for the other columns in the clust_col list too.
Next, created a copy of clust_col called clust_data where missing values (nan) are replaced with “Missing”.

```
clust_data = data[clust_col].copy()
# Treating missing product search method value as Missing
clust_data['Product_Search_Method'] = (
    clust_data['Product_Search_Method'].fillna('Missing'))
print(clust_data.shape)
print(clust_data.isnull().sum())
```

```
(800, 15)
Purchase_Frequency                                0
Browsing_Frequency                                0
Product_Search_Method                             0
Search_Result_Exploration                         0
Add_to_Cart_Browsing                              0
Cart_Completion_Frequency                         0
Saveforlater_Frequency                            0
Recommendation_Helpfulness                        0
Review_Left                                       0
Review_Reliability                                0
Review_Helpfulness                                0
Customer_Reviews_Importance                       0
Personalized_Recommendation_Frequency_Rating      0
Rating_Accuracy                                   0
```

We set the ordinal, nominal and numeric value columns. Ordinal values are those which have a specific rank or order while nominal has no specific order.

```
# Setting up Ordinal (Categories with rank/order), Nominal (No specific order)
# and Numeric columns
from sklearn.preprocessing import OrdinalEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ordinal_col = ['Purchase_Frequency','Browsing_Frequency','Search_Result_Exploration',
               'Add_to_Cart_Browsing','Cart_Completion_Frequency','Saveforlater_Frequency',
               'Recommendation_Helpfulness','Review_Left','Review_Reliability',
               'Review_Helpfulness']
nominal_col = ['Product_Search_Method']
numeric_col = ['Customer_Reviews_Importance','Personalized_Recommendation_Frequency_Rating',
               'Rating_Accuracy','Shopping_Satisfaction']
```

Encoder is used for this

```
ordinal_cat = [['Less than once a month', 'Once a month', 'Few times a month',
                'Once a week', 'Multiple times a week'],
                 ['Rarely', 'Few times a month', 'Few times a week',
                  'Multiple times a day'],
                ['First page', 'Multiple pages'], ['No', 'Maybe', 'Yes'],
                ['Never', 'Rarely', 'Sometimes', 'Often', 'Always'],
                ['Never', 'Rarely', 'Sometimes', 'Often', 'Always'],
                ['No', 'Sometimes', 'Yes'], ['No', 'Yes'],
                ['Never', 'Rarely', 'Occasionally', 'Moderately', 'Heavily'],
                ['No', 'Sometimes', 'Yes']]
og_encoder = OrdinalEncoder(categories = ordinal_cat)
encoded_ordinal = og_encoder.fit_transform(clust_data[ordinal_col])
print(encoded_ordinal.shape)
```

```
(800, 10)
```

```
oneh_encoder = OneHotEncoder(sparse_output = False, handle_unknown = 'ignore')
encoded_nominal = oneh_encoder.fit_transform(clust_data[nominal_col])
print(encoded_nominal.shape)
print(oneh_encoder.get_feature_names_out(nominal_col))
```

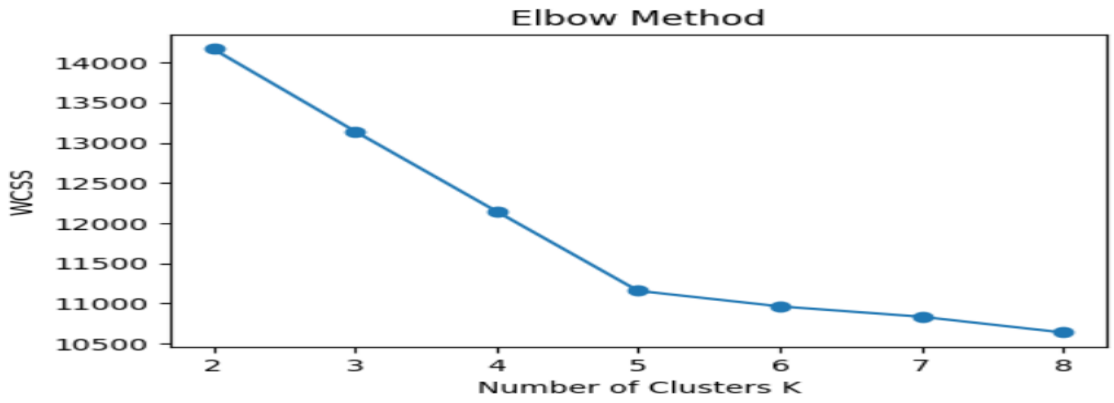
```
(800, 5)
['Product_Search_Method_Filter' 'Product_Search_Method_Keyword'
 'Product_Search_Method_Missing' 'Product_Search_Method_categories'
 'Product_Search_Method_others']
```

```
# Therefore, 10 ordinal, 5 nominal and 4 numeric features
# K means uses distances and scaling done to prevent disproportionate influence
# of any variable on clusters
import numpy as np
encoded_numeric = clust_data[numeric_col].to_numpy()
clust_matrix = np.hstack([encoded_ordinal, encoded_nominal, encoded_numeric])
print('Before Scaling = ', clust_matrix.shape)
scaler = StandardScaler()
clust_scaled = scaler.fit_transform(clust_matrix)
print('After Scaling = ', clust_scaled.shape)
```

```
Before Scaling =  (800, 19)
After Scaling =  (800, 19)
```

Now the value of K is determined using both elbow method and silhouette score.

```
# Determining K (number of clusters) using Elbow method
from sklearn.cluster import KMeans
wcss = []
for k in range(2, 9):
    kmeans = KMeans(n_clusters = k, random_state = 42, n_init = 10)
    kmeans.fit(clust_scaled)
    wcss.append(kmeans.inertia_)
plt.figure(figsize = (5.5, 3.5))
plt.plot(range(2, 9), wcss, marker = 'o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters K')
plt.ylabel('WCSS')
plt.xticks(range(2, 9))
plt.show()
```




```
# Confirming K = 5 with Silhouette Score
from sklearn.metrics import silhouette_score
s_score = []
for k in range(2, 9):
  kmeans = KMeans(n_clusters = k, random_state = 42, n_init = 10)
  labels = kmeans.fit_predict(clust_scaled)
  score = silhouette_score(clust_scaled, labels)
  s_score.append(score)
silhouette_summary = pd.DataFrame({'K': range(2, 9),
                                   'Silhouette Score': s_score})
print(silhouette_summary)
```

```
   K  Silhouette Score
0  2          0.065612
1  3          0.099222
2  4          0.132773
3  5          0.166573
4  6          0.132792
5  7          0.114086
6  8          0.098310
```

Number of clusters come out to be 5 using both elbow method and silhouette score.

Now K-Means model, principal components and cluster profile are created. Cluster profile is then converted to dataframe using pd.DataFrame() function.

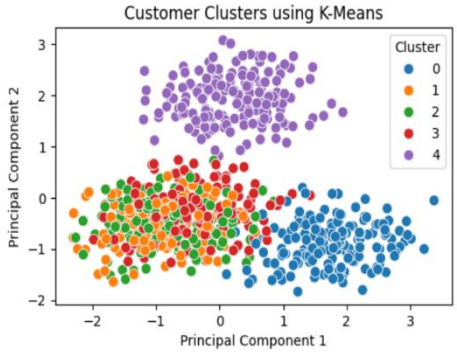
```
# Creating K-Means Model with K = 5
final_kmeans = KMeans(n_clusters = 5, random_state = 42, n_init = 10)
data['Cluster'] = final_kmeans.fit_predict(clust_scaled)
print(data['Cluster'].value_counts().sort_index())
```

```
Cluster
0    163
1    149
2    150
3    156
4    182
Name: count, dtype: int64
```

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
clust_pca = pca.fit_transform(clust_scaled)
pca_df = pd.DataFrame(clust_pca, columns=['PC1', 'PC2'])
pca_df['Cluster'] = data['Cluster']
print(pca_df.head())
print("Explained Variance:", pca.explained_variance_ratio_)
print("Total Explained Variance:", pca.explained_variance_ratio_.sum())
```

```
        PC1       PC2  Cluster
0  0.803729  1.997723        4
1  1.144100  0.073154        3
2  2.293682 -1.392202        0
3  0.041522  1.915133        4
4 -0.535910 -0.478514        2
Explained Variance: [0.07282933 0.06970435]
Total Explained Variance: 0.1425336784844506
```

```
plt.figure(figsize = (5.5, 3.5))
sns.scatterplot(data = pca_df, x = 'PC1', y = 'PC2', hue = 'Cluster',
                palette='tab10', s=60)
plt.title('Customer Clusters using K-Means')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title='Cluster')
plt.show()
```




```
clust_profile = data.groupby('Cluster')[['Purchase_Frequency','Browsing_Frequency',
                                         'Cart_Completion_Frequency',
                                         'Shopping_Satisfaction']].agg(lambda x: x.mode()[0])
print(clust_profile)
clust_size = data['Cluster'].value_counts().sort_index()
print("Cluster Size (%)")
print((clust_size / len(data) * 100).round(2))
```

Cluster	Purchase Frequency	Browsing Frequency	Cart Completion	Shopping Satisfaction	Cluster Size (%)
0	Once a week	Few times a week	Often	1	20.375
1	Once a month	Multiple times a day	Often	5	18.625
2	Multiple times a week	Few times a month	Sometimes	5	18.750
3	Once a month	Rarely	Often	3	19.500
4	Multiple times a week	Few times a week	Always	2	22.750

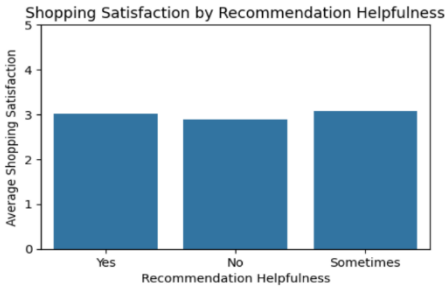
Task 4: Recommendation and Review Insights

1. Examined the relationship between recommendation helpfulness and shopping satisfaction
   Grouped shopping satisfaction by recommendation helpfulness and calculated the mean, median and mode for the values. Found out that the average shopping satisfaction was the higher for recommendation helpfulness as yes or sometimes while it was the lowest for recommendation helpfulness as No.

```
# TASK 4: RECOMMENDATION & REVIEW INSIGHTS

# Task 4.1: Examining the relationship between recommendation helpfulness
#           and shopping satisfaction
# Whether shopping satisfaction depends on how helpful the recommendations are
recommend_s = data.groupby('Recommendation_Helpfulness')[
    'Shopping_Satisfaction'].agg(['mean', 'median', 'count']).round(2)
print(recommend_s)
```

Recommendation_Helpfulness	mean	median	count
No	2.89	3.0	295
Sometimes	3.08	3.0	244
Yes	3.02	3.0	261

```
plt.figure(figsize = (5.5, 3.5))
sns.barplot(data = data, x = 'Recommendation_Helpfulness',
            y = 'Shopping_Satisfaction', errorbar = None)
plt.title('Shopping Satisfaction by Recommendation Helpfulness')
plt.xlabel('Recommendation Helpfulness')
plt.ylabel('Average Shopping Satisfaction')
plt.ylim(0, 5)
plt.show()
```

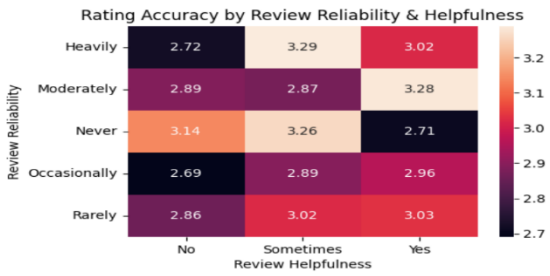


2. Evaluated how review reliability and helpfulness impact overall ratings.
   Grouped rating accuracy by review reliability and review helpfulness and then created a heatmap.

```
# Task 4.2: Evaluating how review reliability
#           and helpfulness impact overall ratings
# How rating accuracy is affected by review reliability & helpfulness
review_rate = data.groupby(['Review_Reliability', 'Review_Helpfulness'])[
    'Rating_Accuracy'].agg(['mean', 'median', 'count']).round(2)
print(review_rate)
```

Review_Reliability	Review_Helpfulness	mean	median	count
Heavily	No	2.72	2.0	57
	Sometimes	3.29	3.0	45
	Yes	3.02	3.0	58
Moderately	No	2.89	3.0	54
	Sometimes	2.87	3.0	62
	Yes	3.28	3.0	39
Never	No	3.14	3.0	50
	Sometimes	3.26	3.0	50
	Yes	2.71	2.0	48
Occasionally	No	2.69	2.0	55
	Sometimes	2.89	3.0	56
	Yes	2.96	3.0	52
Rarely	No	2.86	3.0	57
	Sometimes	3.02	3.0	53
	Yes	3.03	3.0	64

```
r_heatmap = data.pivot_table(values = 'Rating_Accuracy',
                             index = 'Review_Reliability',
                             columns = 'Review_Helpfulness',
                             aggfunc = 'mean')
plt.figure(figsize = (5.5, 3.5))
sns.heatmap(r_heatmap, annot = True, fmt = '.2f')
plt.title('Rating Accuracy by Review Reliability & Helpfulness')
plt.xlabel('Review Helpfulness')
plt.ylabel('Review Reliability')
plt.show()
```



3. Identified trends in how often customers engage with or trust personalized recommendations.
   Found out the contribution of No/Yes/Sometimes in Personalized_Recommendation_Frequency using value_counts() function. Grouped Personalized_Recommendation_Frequency_Rating by Personalized_Recommendation_Frequency and calculated its mean, median and count.

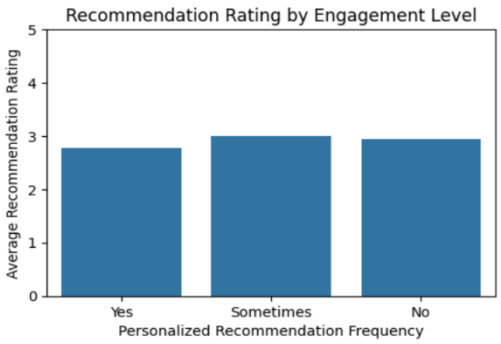
```
# Task 4.3: Identifying trends in how often customers engage with or trust
#           personalized recommendations

# Personalized Recommendation Frequency Value_Counts
recommend_freq = data['Personalized_Recommendation_Frequency'].value_counts()
print(recommend_freq)
print('Percentage')
print((recommend_freq / len(data) * 100).round(2))
```

```
Personalized_Recommendation_Frequency
No            285
Sometimes     272
Yes           243
Name: count, dtype: int64
Percentage
Personalized_Recommendation_Frequency
No            35.62
Sometimes     34.00
Yes           30.38
Name: count, dtype: float64
```

```
# Personalized Recommendation Frequency Rating across recommendations
recommend_freq_rate = data.groupby('Personalized_Recommendation_Frequency')[
    'Personalized_Recommendation_Frequency_Rating'].agg(['mean',
                                                         'median', 'count']).round(2)
print(recommend_freq_rate)
```

Personalized_Recommendation_Frequency	mean	median	count
No	2.94	3.0	285
Sometimes	3.00	3.0	272
Yes	2.77	3.0	243

```
plt.figure(figsize = (5.5, 3.5))
sns.barplot(data = data, x = 'Personalized_Recommendation_Frequency',
            y = 'Personalized_Recommendation_Frequency_Rating',
            errorbar = None)
plt.title('Recommendation Rating by Engagement Level')
plt.xlabel('Personalized Recommendation Frequency')
plt.ylabel('Average Recommendation Rating')
plt.ylim(0, 5)
plt.show()
```



4. Suggested actionable insights for improving Snapdeal's recommendation system.
   i) 35.62% of customers do not engage with personalized recommendations.
      Use behavioural data to make recommendations more relevant.
   ii) Customers engaging consistently with recommendations gave the lowest average ratings of 2.77/5.
      Review relevance and diversity of recommended products
   iii) Customers for whom recommendations are less helpful have lowest shopping satisfaction of 2.89/5.
      Recommend using browsing data and other behavioral data.

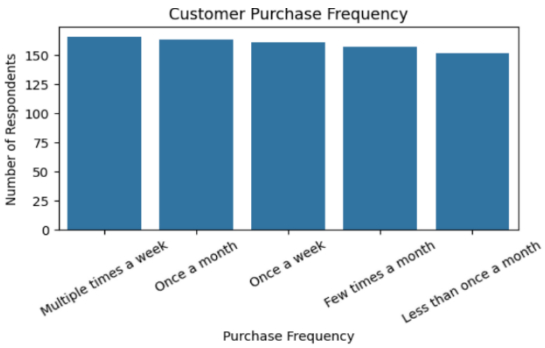
iv) Recommendation strategies should be different for frequent buyers, occasional shoppers and at-risk customers.
Track metrics like cart abandonment rate, add to cart rate, customer satisfaction, cart completion frequency

Task 5: Visualization and Reporting

1. Created attractive visualizations (bar charts, heatmaps, pie charts) for Purchase categories, Browsing frequency distribution, Satisfaction levels, Correlation between recommendation usefulness and satisfaction etc.

i) Purchase Frequency

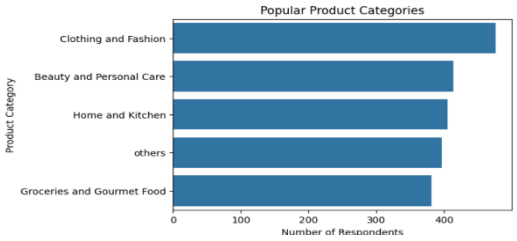
```
# Visualizations
# 1) Purchasing Frequency
plt.figure(figsize = (6, 4))
sns.countplot(data = data, x = 'Purchase_Frequency',
              order = data['Purchase_Frequency'].value_counts().index)
plt.title('Customer Purchase Frequency')
plt.xlabel('Purchase Frequency')
plt.ylabel('Number of Respondents')
plt.xticks(rotation = 30)
plt.tight_layout()
plt.show()
```



Number of respondents for purchase frequency of multiple times a week and once a month are the highest although there does not exist much difference among the number of respondents for different purchase frequencies.

ii) Popular Product Categories

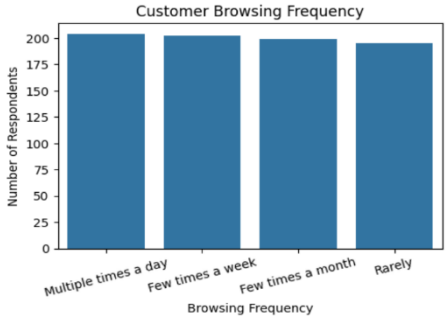
```
# 2) Individual Purchase Categories using cat_count
plt.figure(figsize = (6, 4))
sns.barplot(x = cat_count.values, y = cat_count.index)
plt.title('Popular Product Categories')
plt.xlabel('Number of Respondents')
plt.ylabel('Product Category')
plt.show()
```



The most popular product category is Clothing and Fashion followed by Beauty and Personal Care while the least popular product category is Groceries and Gourmet Food

iii) Browsing Frequency

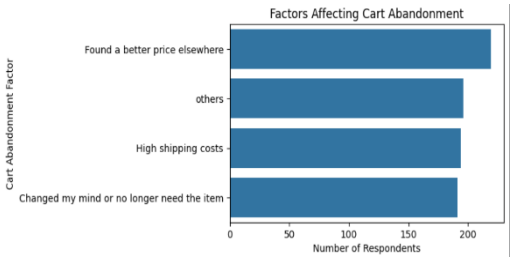
```
# 3) Browsing Frequency
plt.figure(figsize = (5.5, 3.5))
sns.countplot(data = data, x = 'Browsing_Frequency',
              order = data['Browsing_Frequency'].value_counts().index)
plt.title('Customer Browsing Frequency')
plt.xlabel('Browsing Frequency')
plt.ylabel('Number of Respondents')
plt.xticks(rotation = 15)

plt.show()
```



Highest number of respondents are for browsing multiple times a day followed by few times a week.

iv) Cart Abandonment Factors

```
# 4) Cart Abandonment Factors
plt.figure(figsize = (5.5, 3.5))
sns.barplot(x = data['Cart_Abandonment_Factors'].value_counts().values,
            y = data['Cart_Abandonment_Factors'].value_counts().index)
plt.title('Factors Affecting Cart Abandonment')
plt.xlabel('Number of Respondents')
plt.ylabel('Cart Abandonment Factor')
plt.show()
```



Finding a better price elsewhere is the leading cause of cart abandonment followed by high shipping charges.

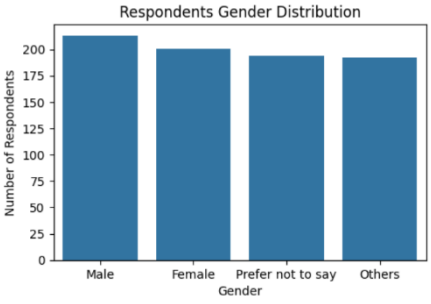
v) Key Rating Metrics

```
# 5) Key Rating Metrics
rating_mean = data[['Shopping_Satisfaction', 'Rating_Accuracy',
                    'Personalized_Recommendation_Frequency_Rating']].mean()
rating_mean.index = ['Shopping Satisfaction', 'Rating Accuracy',
                     'Recommendation Rating']
plt.figure(figsize = (5.5, 3.5))
sns.barplot(x = rating_mean.index, y = rating_mean.values)
plt.title('Avg Customer Ratings')
plt.xlabel('Rating Metric')
plt.ylabel('Avg Rating (1-5)')
plt.ylim(0, 5)
plt.xticks(rotation = 15)
plt.show()
```

Average rating is almost equal for shopping satisfaction, rating accuracy and recommendation rating.

vi) Gender Distribution

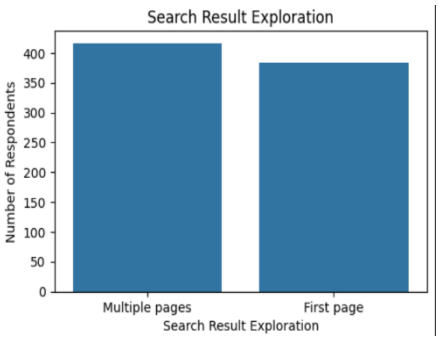
```
# 6) Gender Distribution
plt.figure(figsize = (5.5, 3.5))
sns.countplot(data = data, x = 'Gender',
              order = data['Gender'].value_counts().index)
plt.title('Respondents Gender Distribution')
plt.xlabel('Gender')
plt.ylabel('Number of Respondents')
plt.show()
```



Among the respondents, males are the highest in number followed by females.

vii) Search Result Exploration

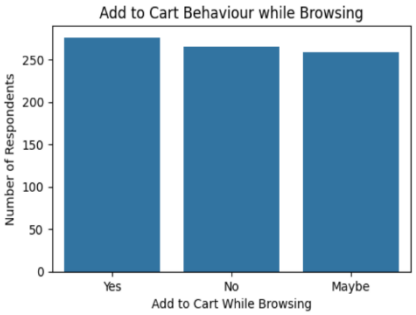
```
# 7) Search Result Exploration
plt.figure(figsize = (5.5, 3.5))
sns.countplot(data = data, x = 'Search_Result_Exploration',
              order = data['Search_Result_Exploration'].value_counts().index)
plt.title('Search Result Exploration')
plt.xlabel('Search Result Exploration')
plt.ylabel('Number of Respondents')
plt.show()
```



Higher number of customers search multiple pages for result exploration.

viii) Add to Cart Browsing

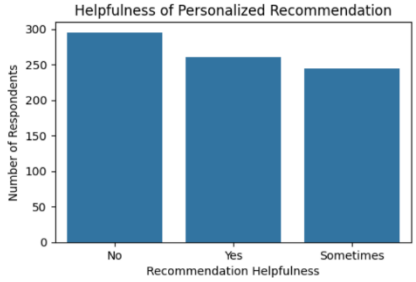
```
# 8) Add to Cart Browsing
plt.figure(figsize = (5.5, 3.5))
sns.countplot(data = data, x = 'Add_to_Cart_Browsing',
              order = data['Add_to_Cart_Browsing'].value_counts().index)
plt.title('Add to Cart Behaviour while Browsing')
plt.xlabel('Add to Cart While Browsing')
plt.ylabel('Number of Respondents')
plt.show()
```



Higher number of customers add products to cart while browsing.

ix) Recommendation Helpfulness

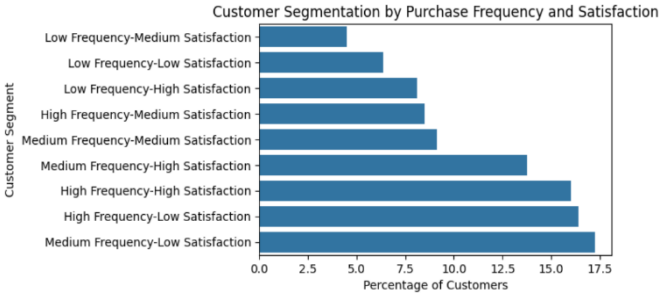
```
# 9) Recommendation Helpfulness
plt.figure(figsize = (5.5, 3.5))
sns.countplot(data = data, x = 'Recommendation_Helpfulness',
              order = data['Recommendation_Helpfulness'].value_counts().index)
plt.title('Helpfulness of Personalized Recommendation')
plt.xlabel('Recommendation Helpfulness')
plt.ylabel('Number of Respondents')
plt.show()
```



Less number of respondents find the personalized recommendations helpful.

x) Customer Segmentation by Purchase Frequency & Satisfaction Levels

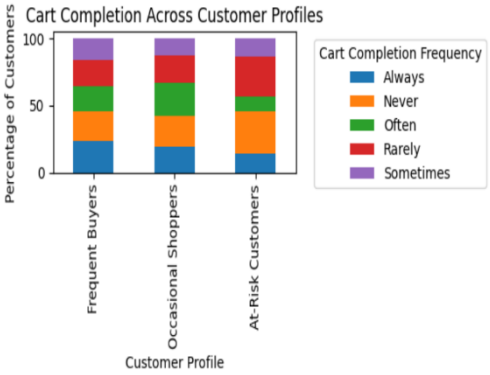
```
plt.figure(figsize=(5.5, 3.5))
sns.barplot(x = segment_summary['Percentage'], y = segment_summary.index)
plt.title('Customer Segmentation by Purchase Frequency and Satisfaction')
plt.xlabel('Percentage of Customers')
plt.ylabel('Customer Segment')
plt.show()
```



There are more medium frequency-low satisfaction customers. Also, low satisfaction customers alone make up around 40% of the total customers.

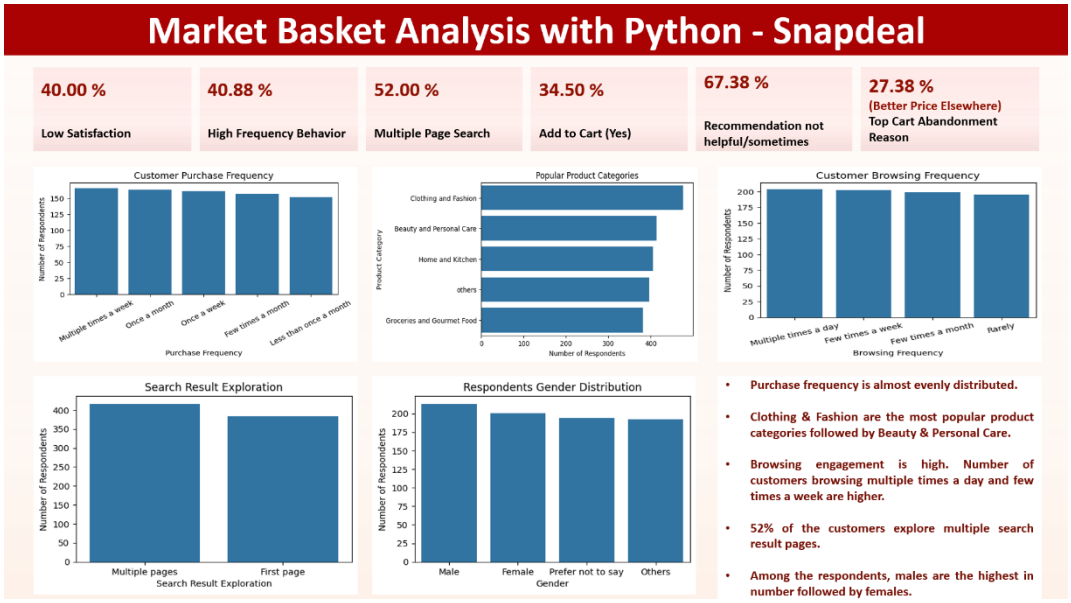
xi) Cart Completion across Customer Profiles

```
cart_profile.T.plot(kind='bar', stacked=True, figsize=(6, 3.5))
plt.title('Cart Completion Across Customer Profiles')
plt.xlabel('Customer Profile')
plt.ylabel('Percentage of Customers')
plt.legend(title='Cart Completion Frequency', bbox_to_anchor=(1.05, 1),
           loc='upper left')
plt.tight_layout()
plt.show()
```



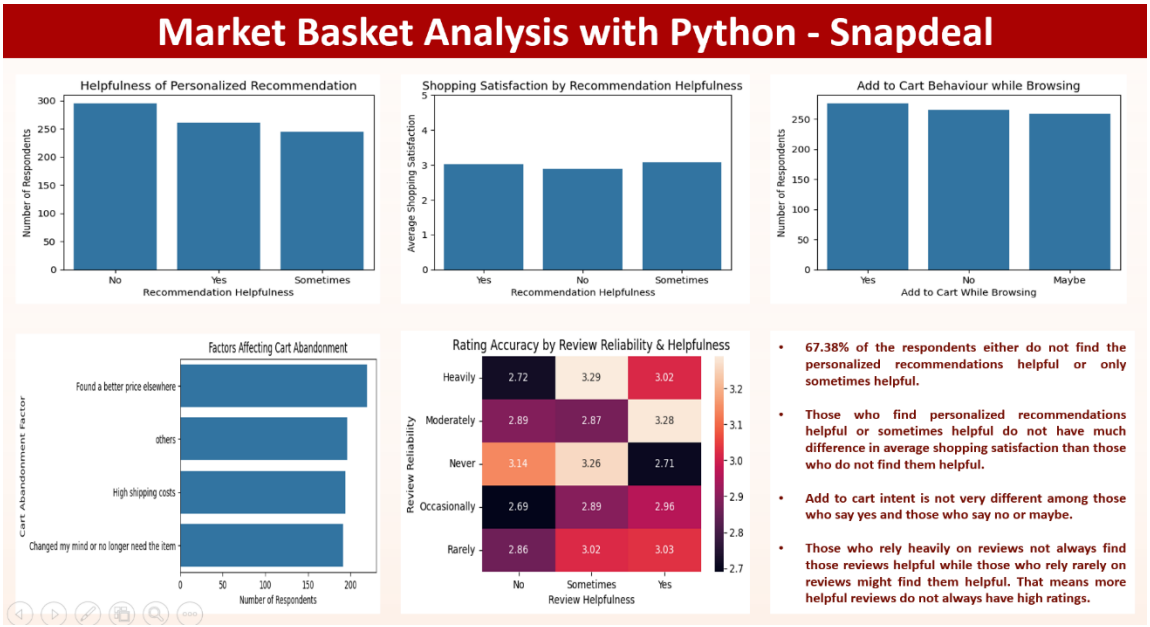
Frequent buyers have higher always/often cart completion percentage of customers while at-risk customers have higher never/rarely percentage of customers.

Insights and Recommendations



Insights for Page 1:

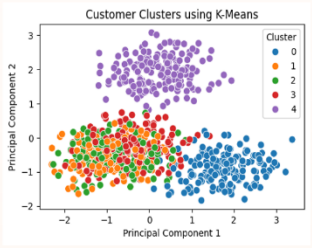
- Purchase frequency is almost evenly distributed.
- Clothing & Fashion are the most popular product categories followed by Beauty & Personal Care.
- Browsing engagement is high. Number of customers browsing multiple times a day and few times a week are higher.
- 52% of the customers explore multiple search result pages.
- Among the respondents, males are the highest in number followed by females.
- Low average shopping satisfaction customers contribute to 40% of the total customers.
- Only 34.5% of total respondents add products to cart while browsing.
- 67.38% of respondents either do not find the personalized recommendations helpful or find them useful only sometimes.
- 27.38% of respondents say that their cart abandonment factor was that they found a better price elsewhere.



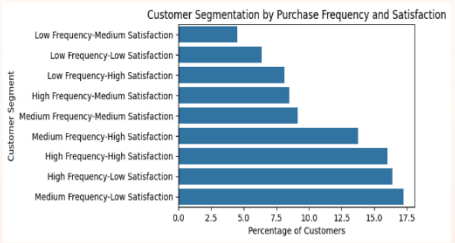
Insights for Page 2:

- 67.38% of the respondents either do not find the personalized recommendations helpful or find them helpful only sometimes.
- Those who find personalized recommendations helpful or sometimes helpful do not have much difference in average shopping satisfaction than those who do not find them helpful.
- Add to cart intent is not very different among those who say yes and those who say no or maybe.
- Major cart abandonment reasons as discussed earlier are better pricing elsewhere and high shipping costs.
- Those who rely heavily on reviews not always find those reviews helpful while those who rely rarely on reviews might find them helpful. That means more helpful reviews do not always have high ratings

Market Basket Analysis with Python - Snapdeal



Cluster	Purchase Frequency	Browsing Frequency	Cart Completion	Shopping Satisfaction	Cluster Size (%)
0	Once a week	Few times a week	Often	1	20.375
1	Once a month	Multiple times a day	Often	5	18.625
2	Multiple times a week	Few times a month	Sometimes	5	18.750
3	Once a month	Rarely	Often	3	19.500
4	Multiple times a week	Few times a week	Always	2	22.750



Cust_Segment	Count	Percentage
Low Frequency-Medium Satisfaction	36	4.500
Low Frequency-Low Satisfaction	51	6.375
Low Frequency-High Satisfaction	65	8.125
High Frequency-Medium Satisfaction	68	8.500
Medium Frequency-Medium Satisfaction	73	9.125
Medium Frequency-High Satisfaction	110	13.750
High Frequency-High Satisfaction	128	16.000
High Frequency-Low Satisfaction	131	16.375
Medium Frequency-Low Satisfaction	138	17.250

- K-Means identified 5 different clusters indicating differences in purchasing, browsing, cart completion and shopping satisfaction patterns.
- Medium frequency & low satisfaction customers form the largest segment (17.25%). This segment also needs retention efforts along with high frequency and low satisfaction segment (16.38%).
- At risk customers have higher never/rarely cart completion frequency while frequent buyers have higher always/often cart completion frequency,
- Customers who browse few times a week and purchase once or multiple times a week have lower shopping satisfaction and form the highest clusters. This group needs better shopping experience.

Insights for Page 3:

- K-Means identified 5 different clusters indicating differences in purchasing, browsing, cart completion and shopping satisfaction patterns.
- Medium frequency & low satisfaction customers form the largest segment (17.25%). This segment also needs retention efforts along with high frequency and low satisfaction segment (16.38%).
- At risk customers have higher never/rarely cart completion frequency while frequent buyers have higher always/often cart completion frequency,
- Customers who browse few times a week and purchase once or multiple times a week have lower shopping satisfaction and form the highest clusters. This group needs better shopping experience.

Market Basket Analysis with Python - Snapdeal

Insights:

- Customers actively browse and 52% of them explore multiple search result pages but add to cart behavior is almost equal for customers who say yes/no/sometimes.
- Finding a better price elsewhere is the highest occurring cart abandonment factor (27.38%) followed by high shipping costs (24.25%).
- 67.38% of the customers either do not find personalized recommendations helpful or find them helpful only sometimes.
- The At-Risk customers have higher never/rarely cart completion, thereby giving a conversion and retention opportunity.
- There exists distinct combinations of customers with different purchase frequency and satisfaction levels. Hence, different segments should be treated with different engagement strategies.
- Customers who browse few times a week and purchase once or multiple times a week have lower shopping satisfaction and form the highest clusters.

Recommendations:

- Use behavioral data of customers like browsing, purchasing, cart abandonment, cart completion etc. to personalize recommendations and ensure the customers find them helpful.
- Monitoring of competitor pricing and using targeted discounts and lowering shipping costs are needed to reduce cart abandonment.
- Improve product search exploration and filters so that customers find the required products faster and on the first page.
- Target customers with low satisfaction, high cart abandonment and at-risk customers with low cart completion.
- Improve the shopping experience of customers who browse few times a week and purchase once or multiple times a week (43.13% cluster size) while also encouraging customers who browse multiple times a day but purchase once a month (18.63% cluster size) to buy more.

Recommendations:

- Use behavioral data of customers like browsing, purchasing, cart abandonment, cart completion etc. to personalize recommendations and ensure the customers find them helpful.
- Monitoring of competitor pricing and using targeted discounts and lowering shipping costs are needed to reduce cart abandonment.
- Improve product search exploration and filters so that customers find the required products faster and on the first page.
- Target customers with low satisfaction, high cart abandonment and at-risk customers with low cart completion.
- Improve the shopping experience of customers who browse few times a week and purchase once or multiple times a week (43.13% cluster size) while also encouraging customers who browse multiple times a day but purchase once a month (18.63% cluster size) to buy more.

Video Link:

Link